

Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого

—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 3

«РАБОТА С ТАЙМЕРАМИ-СЧЁТЧИКАМИ»

по дисциплине «Аппаратные средства вычислительной техники»

Выполнил
студент гр.5151001/40001

<подпись>

Волошкевич М.А.

Преподаватель /
ассистент

<подпись>

Семенов П.О.

Санкт-Петербург
2026г.

1. Цель работы.

Получение практических навыков по работе с таймерами-счётчиками и применению механизма прерываний.

2. Постановка задачи

Модуль проверки ПИН-кода. Программа должна предоставлять возможность по проверке введённого ПИН-кода на соответствие заданному. ПИН-код должен храниться в EEPROM и считываться при запуске МК (запись ПИН-кода в EEPROM следует выполнять с помощью AVRFlash). Для отображения проверяемого ПИН-кода используется блок из четырёх семисегментных индикаторов. Для ввода цифры ПИНкода используются кнопки, подключённые к PORTA и PORTB: PB0 – цифра 0, PB1 – 1, PB2 – 2, PB3 – 3, PB4 – 4, PB5 – 5, PB6 – 6, PB7 – 7, PA4 – 8, PA5 – 9. При вводе неправильного ПИН-кода загорается светодиод PA6 на 20 секунд, после чего программа возвращается к началу ввода ПИН-кода для проверки. При вводе неправильного ПИН-кода три раза подряд загораются светодиод PA7 и PA6, а программа перестаёт работать. При вводе правильного ПИН-кода загорается светодиод PA7, вернуться к началу режима проверки ПИН-кода (завершить текущий сеанс) можно с помощью прерывания INT1 (кнопка PD3) – выполняется очистка семисегментных индикаторов и погаснет светодиод PA7. Ввод четырёхзначного ПИН-кода осуществляется поразрядно, если в течение 7 секунд с момента ввода 1-3 цифры ПИН-кода не была введена следующая цифра программа возвращается к началу ввода ПИНкода для проверки. Ввод ПИН-кода осуществляется от младшего разряда к старшему. В начале работы программы на семисегментном индикаторе нет цифр, сразу после ввода каждой цифры она дописывается на семисегментный индикатор.

*Сделать выбор цифр не только через кнопки, но и через потенциометр

3. Ход работы

3.1. Схема установки

Схема установки с задействованными узлами отладочной платы показана на Рис. 1.

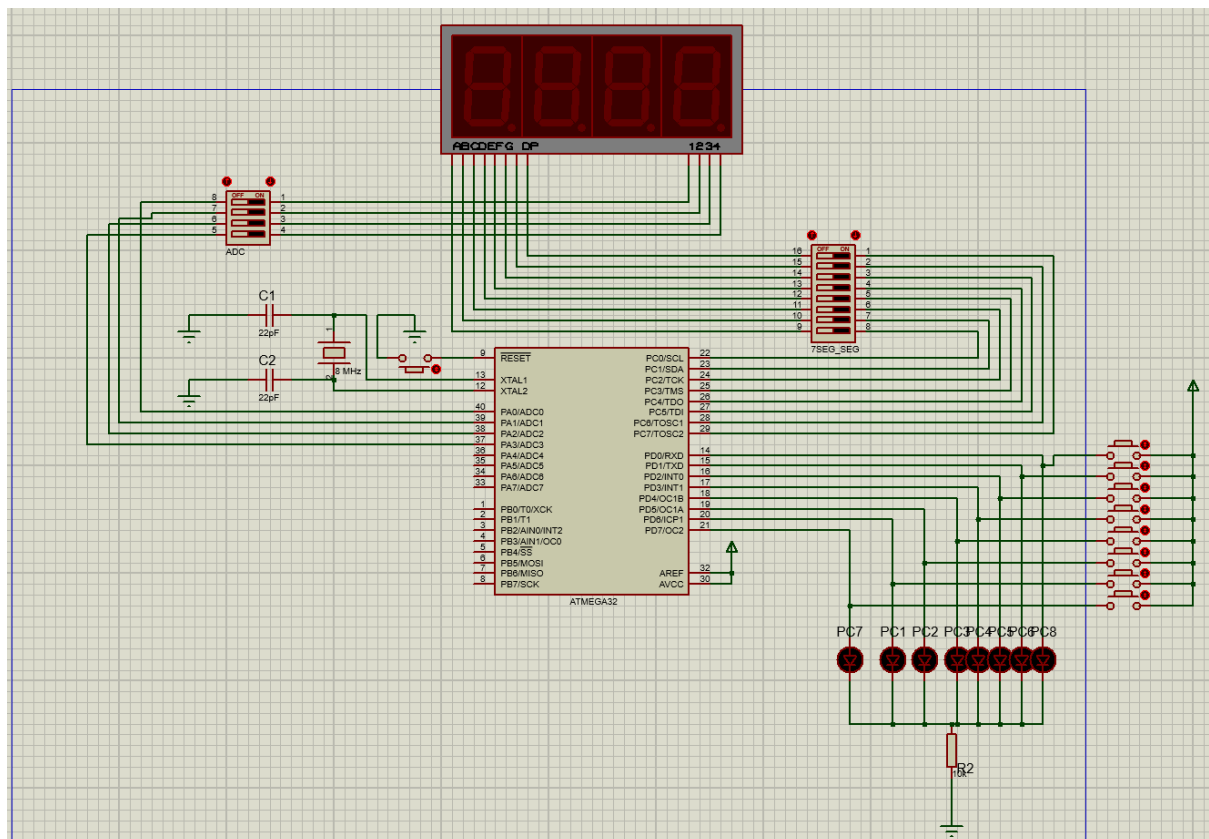


Рис. 1. Схема установки.

3.2. Блок схема работы программы

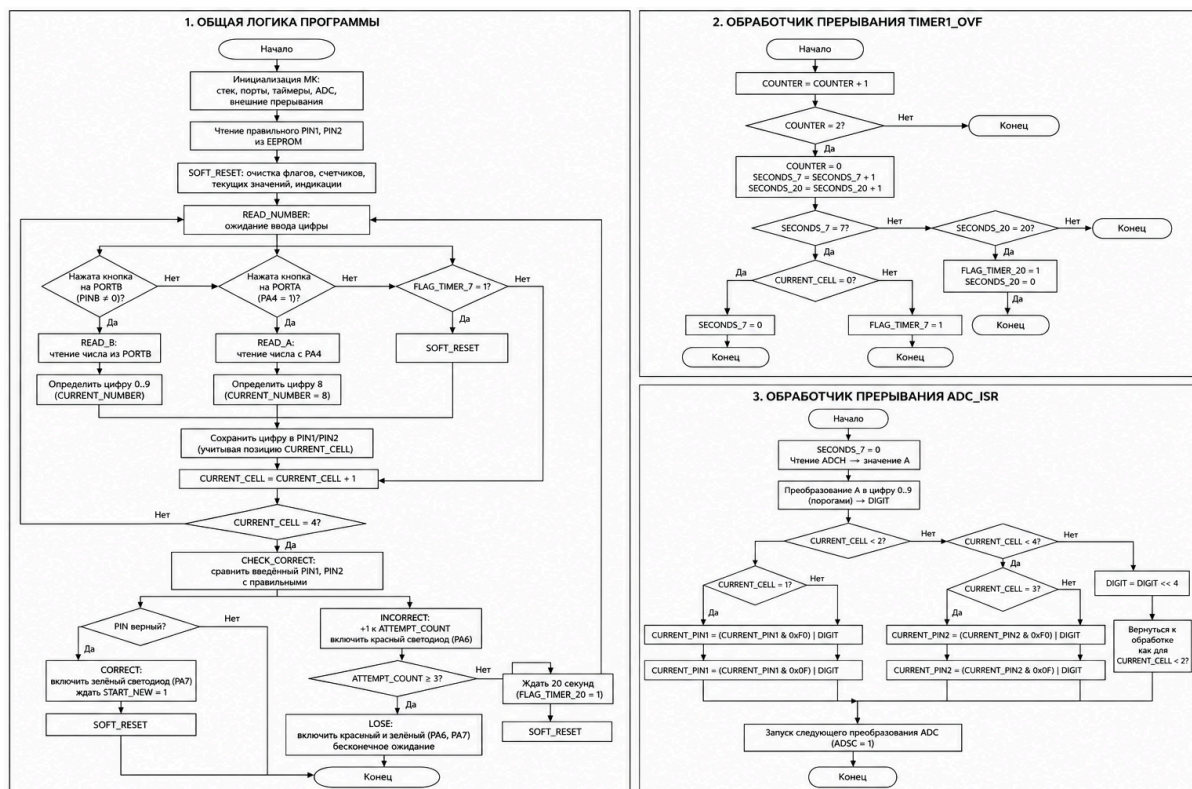


Рис. 2. Блок схема работы программы

3.3. Ответы на контрольные вопросы

1. Посредством каких регистров производится конфигурирование таймера-счётчика? Конфигурирование таймера-счётчика в микроконтроллерах семейства AVR осуществляется с использованием нескольких специализированных регистров. Основным регистром управления является регистр TCCR0, который задаёт режим работы таймера, источник тактового сигнала и коэффициент предделителя. Текущее значение счётчика хранится в регистре TCNT0, который также может быть использован для задания начального значения таймера. Управление прерываниями осуществляется через регистр TIMSK, в котором включаются или отключаются соответствующие источники прерываний. Для фиксации факта возникновения прерывания используется регистр флагов TIFR. В режимах сравнения и широтно-импульсной модуляции применяется регистр OCR0, который задаёт значение, с которым сравнивается счётчик.
2. Какие источники импульсов могут применяться для увеличения таймера-счётчика и для каких целей? Таймер-счётчик может увеличивать своё значение за счёт различных источников импульсов. Основным источником является внутренний тактовый сигнал микроконтроллера, который используется в режиме таймера для измерения временных интервалов и формирования задержек. Также может применяться внешний сигнал, подаваемый на соответствующий вход микроконтроллера, что реализует режим счётчика и позволяет учитывать внешние события, например импульсы от датчиков. Кроме того, между источником тактового сигнала и таймером может использоваться предделитель, который уменьшает частоту поступающих импульсов и позволяет формировать более длительные временные интервалы.

1. В каких режимах могут работать таймеры-счётчики?

Таймеры-счётчики микроконтроллеров AVR могут работать в нескольких режимах, каждый из которых предназначен для решения определённых задач. В нормальном режиме (Normal) счётчик увеличивается от нуля до максимального значения, после чего

происходит переполнение. В режиме CTC (Clear Timer on Compare Match) таймер автоматически сбрасывается при достижении заданного значения, что удобно для формирования точных временных интервалов. В режимах широтно-импульсной модуляции, таких как Phase Correct PWM и Fast PWM, таймер используется для генерации сигналов с регулируемой скважностью, применяемых, например, для управления яркостью светодиодов или скоростью вращения двигателей.

3. Как рассчитать начальное значение таймера-счётчика по заданному времени, которое должен отмерить таймер-счётчик до своего переполнения? Для того чтобы таймер отсчитал заданный интервал времени до момента переполнения, необходимо правильно выбрать его начальное значение. Расчёт производится исходя из тактовой частоты микроконтроллера и коэффициента делителя. Количество импульсов, которое должно быть отсчитано, определяется как произведение требуемого времени на частоту тактирования, делённую на коэффициент делителя. Поскольку таймер имеет ограниченную разрядность, например 8 бит, его максимальное значение равно 256. Следовательно, начальное значение таймера определяется как разность между максимальным значением и рассчитанным числом отсчётов. Таким образом, таймер переполнится ровно через заданный интервал времени.
4. В чём состоит отличие работы таймера-счётчика в режиме таймера и в режиме счётчика? Основное отличие работы таймера-счётчика в режиме таймера и в режиме счётчика заключается в источнике тактовых импульсов. В режиме таймера увеличение значения счётчика происходит за счёт внутреннего тактового генератора микроконтроллера, что позволяет использовать его для измерения времени и формирования временных задержек. В режиме счётчика увеличение происходит от внешних импульсов, поступающих на специальный вход, что позволяет использовать устройство для подсчёта внешних событий. Таким образом, в режиме таймера устройство измеряет время, а в режиме счётчика — количество поступивших импульсов.

4. Вывод

В ходе выполнения лабораторной работы были изучены принципы работы таймеров-счётчиков микроконтроллера AVR и механизм обработки прерываний. Были рассмотрены режимы работы таймера, способы его конфигурирования, а также особенности формирования временных интервалов с использованием аппаратных средств микроконтроллера. В практической части была разработана программа таймера, обеспечивающая настройку времени, запуск, приостановку и возобновление отсчёта, а также отображение текущего значения времени на четырёхразрядном семисегментном индикаторе. Для управления использовались внешние прерывания и кнопки, а обработка временных интервалов реализована с помощью прерываний таймера-счётчика. В результате выполнения работы были получены навыки работы с регистрами микроконтроллера, настройки таймера и реализации алгоритмов с использованием прерываний.

Приложения

Приложение А

main.asm

```
.include "m32def.inc"

.equ EEPROM_ADR_PIN1 = 0x10
.equ EEPROM_ADR_PIN2 = 0x11

.def ATTEMPT_COUNT = R12
.def FLAG_TIMER_20 = R13
.def FLAG_TIMER_7 = R14
.def START_NEW = R15
.def NULL = R16
.def TMP = R17
.def TMP2 = R18
.def CURRENT_CELL = R19
.def RIGHT_PIN1 = R20
.def RIGHT_PIN2 = R21
.def TIMER_TMP = R22
.def DISP_CELL = R23
.def CURRENT_PIN1 = R24
.def CURRENT_PIN2 = R25
.def CURRENT_NUMBER = R26
.def CURRENT_READ_NUMBER = R27
.def COUNTER = R28
.def SECONDS_7 = R29
.def SECONDS_20 = R30
.def ADC_TMP = R31

.org $000
    JMP reset
.org INT0addr
    JMP ext_int0
.org INT1addr
    JMP ext_int1
.org OVFladdr
    RJMP TIMER1_OVF
```



```

.org 0C0addr
    RJMP TIMER0_COMP
.org ADCCaddr
    RJMP ADC_ISR

```

SEG_TABLE:

```

.db 0b00111111, 0b00000110 ; 0, 1
.db 0b01011011, 0b01001111 ; 2, 3
.db 0b01100110, 0b01101101 ; 4, 5
.db 0b01111101, 0b00000111 ; 6, 7
.db 0b01111111, 0b01101111 ; 8, 9
.db 0b01110111, 0b01111100 ; A, b
.db 0b00111001, 0b01011110 ; C, d
.db 0b01111001, 0b01110001 ; E, F

```

reset:

```

    LDI TMP, HIGH(RAMEND)
    OUT SPH, TMP
    LDI TMP, LOW(RAMEND)
    OUT SPL, TMP

    CLR NULL ; 0x00
    CLR ATTEMPT_COUNT

    LDI TMP, EEPROM_ADR_PIN1
    CALL EEPROM_read
    MOV RIGHT_PIN1, TMP2

    LDI TMP, EEPROM_ADR_PIN2
    CALL EEPROM_read
    MOV RIGHT_PIN2, TMP2

    CLR TMP ; 0x00
    OUT DDRB, TMP
    LDI TMP, 0xCF
    OUT DDRA, TMP
    LDI TMP, 0xF2
    OUT DDRD, TMP
    SER TMP

```

```
OUT DDRC, TMP
```

```
; ?????????????? /1024
```

```
LDI TMP, (1<<CS11)|(1<<CS10)
```

```
OUT TCCR1B, TMP
```

```
;IN TMP, TIMSK
```

```
LDI TMP, (1<<TOIE1)|(1<<OCIE0)
```

```
OUT TIMSK, TMP
```

```
LDI TMP, (1<<WGM01)|(1<<CS01)|(1<<CS00) ; CTC, /64
```

```
OUT TCCR0, TMP
```

```
LDI TMP, 124
```

```
OUT OCR0, TMP
```

```
LDI TMP, 0x65
```

```
OUT ADMUX, TMP
```

```
LDI TMP, 0x8E
```

```
OUT ADCSRA, TMP
```

```
; ?????? ???????? ??????????????????
```

```
SBI ADCSRA, ADSC
```

```
LDI TMP, 0x0F
```

```
OUT MCUCR, TMP
```

```
LDI TMP, 0xC0
```

```
OUT GICR, TMP
```

```
OUT GIFR, TMP
```

```
SEI
```

```
soft_reset:
```

```
CLR FLAG_TIMER_20
```

```
CLR FLAG_TIMER_7
```

```
CLR START_NEW
```

```
CLR SECONDS_7
```

```
CLR SECONDS_20
```

```
CLR COUNTER
```

```
CLR TMP
CLR TMP2
CLR CURRENT_CELL
CLR CURRENT_PIN1
CLR CURRENT_PIN2
CLR CURRENT_NUMBER
CLR CURRENT_READ_NUMBER
CLR START_NEW
OUT PORTA, TMP
```

```
IN  TMP, PORTA
ANDI TMP, 0b11000000 ; PA0-PA3 = LOW, PA6/PA7 ?? ???????
OUT  PORTA, TMP
OUT  PORTC, NULL
RJMP read_number
```

```
check_correct:
    CP CURRENT_PIN1, RIGHT_PIN1
    BRNE incorrect
    CP CURRENT_PIN2, RIGHT_PIN2
    BRNE incorrect

    RJMP correct
```

```
incorrect:
    INC ATTEMPT_COUNT
    MOV TMP, ATTEMPT_COUNT
    CPI TMP, 0x03
    BRSH lose
```

```
CLR FLAG_TIMER_20
CLR TMP
OUT TCNT1H, TMP
OUT TCNT1L, TMP
CLR SECONDS_20
CLR COUNTER
```

```
IN  TMP, PORTA
ORI  TMP, (1<<PA6)
OUT  PORTA, TMP
```

```

incorrect_loop:
    MOV TMP, FLAG_TIMER_20
    CPI TMP, 1
    BRNE incorrect_loop

    CLR FLAG_TIMER_20

    RJMP soft_reset

correct:
    LDI TMP, (1 << PA7)
    OUT PORTA, TMP
wait_new:
    MOV TMP, START_NEW
    CPI TMP, 1
    BRNE wait_new
    RJMP soft_reset

lose:
    LDI TMP, (1 << PA7)
    ORI TMP, (1 << PA6)

    OUT PORTA, TMP
    RJMP lose

check_7sec:
    MOV TMP, FLAG_TIMER_7
    CPI TMP, 1
    BREQ rjmp_soft_reset

    RJMP read_number

rjmp_soft_reset:
    RJMP soft_reset

read_number:
    CPI CURRENT_CELL, 4
    BREQ check_correct

```

```
IN TMP, PINB
IN TMP2, PINA
ANDI TMP2, 0b00010000
```

```
CPSE TMP, NULL
RJMP read_b
```

```
CPSE TMP2, NULL
RJMP read_a
```

```
RJMP check_7sec
```

read_b:

```
NOP
NOP
NOP
NOP
```

```
IN TMP, PINB
CP TMP, NULL
BREQ read_number
```

```
CLR SECONDS_7
CLR COUNTER
```

```
MOV CURRENT_READ_NUMBER, TMP
```

stop_reading_b:

```
IN TMP, PINB
CP TMP, NULL
BRNE stop_reading_b
CLR CURRENT_NUMBER
RJMP get_number_b
```

read_a:

```
NOP
NOP
NOP
NOP
```

```
IN TMP2, PINA
```

```

    ANDI TMP2, 0b00010000
    CP TMP2, NULL
    BREQ read_number

    CLR SECONDS_7
    CLR COUNTER
    SBRC TMP2, 4
    LDI CURRENT_NUMBER, 0x08
stop_reading_a:
    IN TMP2, PINA
    ANDI TMP2, 0b00010000
    CP TMP2, NULL
    BRNE stop_reading_a
    RJMP set_number

inc_current_number:
    LSR CURRENT_READ_NUMBER
    INC CURRENT_NUMBER

get_number_b:
    OUT PORTC, CURRENT_READ_NUMBER
    SBRS CURRENT_READ_NUMBER, 0
    RJMP inc_current_number

set_number:
    INC CURRENT_CELL
    CPI CURRENT_CELL, 3
    BRPL set_34

    CPI CURRENT_CELL, 2
    BREQ shift_number

write_12:
    OR CURRENT_PIN1, CURRENT_NUMBER
    RJMP read_number

set_34:
    CPI CURRENT_CELL, 4
    BREQ shift_number

write_34:
    OR CURRENT_PIN2, CURRENT_NUMBER

```

```
RJMP read_number
```

```
shift_number:
```

```
    LSL CURRENT_NUMBER
```

```
    LSL CURRENT_NUMBER
```

```
    LSL CURRENT_NUMBER
```

```
    LSL CURRENT_NUMBER
```

```
CPI CURRENT_CELL, 3
```

```
    BRPL write_34
```

```
RJMP write_12
```

```
update_current_cell:
```

```
    CLR CURRENT_CELL
```

```
    RJMP return_to_ext0
```

```
ext_int0:
```

```
    PUSH ADC_TMP
```

```
    PUSH ADC_TMP
```

```
    IN ADC_TMP, SREG
```

```
    INC CURRENT_CELL
```

```
CPI CURRENT_CELL, 4
```

```
BREQ update_current_cell
```

```
return_to_ext0:
```

```
    POP ADC_TMP
```

```
    OUT SREG, ADC_TMP
```

```
    POP ADC_TMP
```

```
    RETI
```

```
ext_int1:
```

```
    PUSH TMP
```

```
    IN TMP, SREG
```

```
    INC START_NEW
```

```
OUT SREG, TMP
POP TMP
RETI
```

TIMER1_OVF:

```
INC COUNTER
CPI COUNTER, 2
BRNE OVF_EXIT
```

```
CLR COUNTER
```

```
INC SECONDS_7
INC SECONDS_20
```

```
CPI SECONDS_7, 7
BREQ set_flag_7
```

check_20:

```
CPI SECONDS_20, 20
BREQ set_flag_20
```

```
RJMP OVF_EXIT
```

set_flag_7:

```
CPI CURRENT_CELL, 0
BRNE cur_cel_0
CLR SECONDS_7
```

```
RJMP check_20
```

cur_cel_0:

```
INC FLAG_TIMER_7
RJMP OVF_EXIT
```

set_flag_20:

```
INC FLAG_TIMER_20
CLR SECONDS_20
```

OVF_EXIT:

```
RETI
```


TIMER0_COMP:

```
PUSH    TIMER_TMP
    IN    TIMER_TMP, SREG                ; ?????????? ??????
PUSH    TIMER_TMP
PUSH    ZL
PUSH    ZH

    IN    TIMER_TMP, PORTA
    ANDI  TIMER_TMP, 0b11100000
    OUT   PORTA, TIMER_TMP
;OUT    PORTC, NULL

    CPI   DISP_CELL, 0
    BREQ  d1
    CPI   DISP_CELL, 1
    BREQ  d2
    CPI   DISP_CELL, 2
    BREQ  d3
    CPI   DISP_CELL, 3
    BREQ  d4

    RJMP  OC0_EXIT
```

d1:

```
IN    TIMER_TMP, PORTA
    ORI   TIMER_TMP, (1<<PA3)
    OUT   PORTA, TIMER_TMP

    MOV   TIMER_TMP, CURRENT_PIN1
    ANDI  TIMER_TMP, 0x0F
    RJMP  display_show_digit
```

d2:

```
IN    TIMER_TMP, PORTA
    ORI   TIMER_TMP, (1<<PA2)
    OUT   PORTA, TIMER_TMP

    MOV   TIMER_TMP, CURRENT_PIN1
    LSR   TIMER_TMP
```

```
LSR  TIMER_TMP
LSR  TIMER_TMP
LSR  TIMER_TMP
RJMP display_show_digit
```

d3:

```
IN    TIMER_TMP, PORTA
ORI   TIMER_TMP, (1<<PA1)
OUT   PORTA, TIMER_TMP

MOV   TIMER_TMP, CURRENT_PIN2
ANDI  TIMER_TMP, 0x0F
RJMP  display_show_digit
```

d4:

```
IN    TIMER_TMP, PORTA
ORI   TIMER_TMP, (1<<PA0)
OUT   PORTA, TIMER_TMP

MOV   TIMER_TMP, CURRENT_PIN2
LSR   TIMER_TMP
LSR   TIMER_TMP
LSR   TIMER_TMP
LSR   TIMER_TMP
RJMP  display_show_digit
```

display_show_digit:

```
LDI   ZL, low(SEG_TABLE*2)
LDI   ZH, high(SEG_TABLE*2)
ADD   ZL, TIMER_TMP
BRCC  DSH_NO_CARRY
INC   ZH
```

DSH_NO_CARRY:

```
LPM   TIMER_TMP, Z
OUT   PORTC, TIMER_TMP
RJMP  OC0_EXIT
```

OC0_EXIT:

```
INC   DISP_CELL
ANDI  DISP_CELL, 0x03
```

```
POP    ZH
POP    ZL
POP    TIMER_TMP
OUT    SREG, TIMER_TMP        ; ?????????????????? ?????
POP    TIMER_TMP
```

```
RETI
```

```
EEPROM_read:
```

```
SBIC EECR, EEWE
RJMP EEPROM_read
```

```
OUT EEARL, TMP
OUT EEARH, NULL
SBI EECR, EERE
IN TMP2, EEDR
```

```
RET
```

```
ADC_ISR:
```

```
PUSH ADC_TMP
IN ADC_TMP, SREG
PUSH ADC_TMP
```

```
CLR SECONDS_7
IN ADC_TMP, ADCH
```

```
CPI ADC_TMP, 25
BRLO set_0
```

```
CPI ADC_TMP, 51
BRLO set_1
```

```
CPI ADC_TMP, 76
BRLO set_2
```

```
CPI ADC_TMP, 102
BRLO set_3
```

```
CPI  ADC_TMP, 127
    BRL0 set_4
```

```
CPI  ADC_TMP, 153
    BRL0 set_5
```

```
CPI  ADC_TMP, 178
    BRL0 set_6
```

```
CPI  ADC_TMP, 204
    BRL0 set_7
```

```
CPI  ADC_TMP, 229
    BRL0 set_8
```

```
RJMP set_9
```

```
set_0:
    LDI ADC_TMP, 0x00
    RJMP write_num_adc
```

```
set_1:
    LDI ADC_TMP, 0x01
    RJMP write_num_adc
```

```
set_2:
    LDI ADC_TMP, 0x02
    RJMP write_num_adc
```

```
set_3:
    LDI ADC_TMP, 0x03
    RJMP write_num_adc
```

```
set_4:
    LDI ADC_TMP, 0x04
    RJMP write_num_adc
```

```
set_5:
    LDI ADC_TMP, 0x05
    RJMP write_num_adc
```

```
set_6:
    LDI ADC_TMP, 0x06
    RJMP write_num_adc
```

```
set_7:
```

```

    LDI ADC_TMP, 0x07
    RJMP write_num_adc
set_8:
    LDI ADC_TMP, 0x08
    RJMP write_num_adc
set_9:
    LDI ADC_TMP, 0x09

write_num_adc:
    CPI CURRENT_CELL, 3
    BRPL set_34_adc

    CPI CURRENT_CELL, 2
    BREQ shift_number_adc

write_12_adc:
    CPI CURRENT_CELL, 1
    BREQ andi_cur1
    BRNE andi_cur2

set_after_andi_12:
    OR CURRENT_PIN1, ADC_TMP
    RJMP adc_EXIT
set_34_adc:
    CPI CURRENT_CELL, 4
    BREQ shift_number_adc

write_34_adc:
    CPI CURRENT_CELL, 3
    BREQ andi_cur3
    BRNE andi_cur4

set_after_andi_34:
    OR CURRENT_PIN2, ADC_TMP
    RJMP adc_EXIT

andi_cur1:
    ANDI CURRENT_PIN1, 0xF0
    RJMP set_after_andi_12
andi_cur2:

```

```
    ANDI CURRENT_PIN1, 0x0F
    RJMP set_after_andi_12
```

```
andi_cur3:
    ANDI CURRENT_PIN2, 0xF0
    RJMP set_after_andi_34
```

```
andi_cur4:
    ANDI CURRENT_PIN2, 0x0F
    RJMP set_after_andi_34
```

```
shift_number_adc:
```

```
    LSL ADC_TMP
```

```
    LSL ADC_TMP
```

```
    LSL ADC_TMP
```

```
    LSL ADC_TMP
```

```
    CPI CURRENT_CELL, 3
    BRPL write_34_adc
```

```
    RJMP write_12_adc
```

```
adc_EXIT:
```

```
    SBI  ADCSRA, ADSC
```

```
    POP ADC_TMP
```

```
    OUT  SREG, ADC_TMP
```

```
    POP ADC_TMP
```

```
    RETI
```